



THE UNIVERSITY OF ZAMBIA
SCHOOL OF NATURAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE

NAME: PATRICIA MBOMA KASHWEKA

COMPUTER NUMBER: 2021527433

COURSE CODE: 2021527433

LECTURER: MARTIN PHIRI

Task 1: Requirement Analysis

Functional requirements

- Web-based collection system
- Employees can receive shopping lists on tablets.
- Employee can mark an item as collected or unavailable.
- Employee can hand over a completed list to the payment engine.
- System must work offline on tablets (create, update, and synchronize later).
- Lists are pushed into the collection system via a queue.
- System must notify employees of newly assigned lists (push or local polling).
- Synchronization must be idempotent and conflict resolution-enabled.
- Audit logs for collected/unavailable item actions and list transfers.
- Admin monitoring dashboard (system health, queue backlog, throughput).

Non-Functional Requirements

- Support 200 concurrent employees (employee tablet sessions).
- Process 10,000 shopping lists a day.
- Average shopping list payload $\approx$ 500 KB.
- Annual storage approx 2 TB (including metadata, logs, backups).
- Scalability (horizontal scaling for backend components).
- Offline-first functionality with tight sync and minimal data loss.
- Must be highly available, secure and auditable

Assumptions

- Customers and order creation (checkout) are already executed by the customer system.
- Shopping lists all arrive through an existing message queue.
- Employee tablets are modern with periodic connectivity.
- Payment engine is a separate system with a well-established API.
- Employees use company-provided tablets with standardized browsers.
- Internet connectivity in stores is generally reliable but may have temporary outages.
- Shopping lists contain standard grocery items with barcodes.

- Store managers need basic reporting capabilities.
- System operates during standard retail hours (6AM – 11PM).

Task 2: High-Level Architecture

The GroceColl collection system follows a distributed architecture with offline-first design principles. The system consists of three main tiers:

Presentation Layer

- Progressive Web Application (PWA) running on employee tablets
- Responsive web interface optimized for touch interaction
- Local data caching for offline operations

Application Layer

- RESTful API services handling business logic
- Queue management system for list distribution
- Synchronization services for offline/online data consistency
- Payment integration layer

Data Layer

- Primary database for persistent storage
- Local storage on tablets for offline capability
- File storage for list attachments and images

Key Architectural Decisions

- Offline-First Approach: The system prioritizes local functionality, syncing with servers when connectivity is available. This ensures employees can continue working during network interruptions.
- Microservices Pattern: Different functionalities are separated into distinct services (Lists, Items, Payment, Sync) allowing for independent scaling and maintenance.
- Event-Driven Architecture: Uses message queues for reliable list distribution and status updates, ensuring no data loss during high traffic periods

Task 3: Component Identification

1. Lists Management Service

What it does: Handles shopping list distribution and tracking

Application Type: RESTful API Service

Technology Stack: Node.js with Express framework, Redis for caching

Justification: Node.js provides excellent performance for I/O operations and JSON handling, which is perfect for processing shopping lists.

2. Employee Portal (PWA)

What it does: User interface for employees to interact with shopping lists

Application Type: Progressive Web Application

Technology Stack: React.js with Service Workers, IndexedDB for local storage Justification: PWA enables offline functionality while React provides a responsive, component-based interface ideal for tablet use.

3. Synchronization Service

What it does: Manages data consistency between tablets and central system

Application Type: Background API Service

Technology Stack: Python with Celery for task queuing, PostgreSQL for data persistence Justification: Python excels at data processing tasks, while Celery provides robust background job processing.

4. Payment Integration Service

What it does: Transfers completed collection data to payment processing system Application Type: API Gateway/Adapter Service

Technology Stack: Java Spring Boot with Apache Kafka

Justification: Java provides enterprise-level reliability, Kafka ensures reliable message delivery to payment systems.

5. Queue Management System

What it does: Distributes incoming shopping lists to available employees

Application Type: Message Broker Service

Technology Stack: RabbitMQ with clustering support

Justification: RabbitMQ offers excellent reliability and supports complex routing patterns needed for list distribution.

6. Database cluster

Purpose: Persistent storage for all system data

Application Type: Database Management System

Technology Stack: PostgreSQL with read replicas

Task 4: Technology Stack Selection

Backend Services

Primary Choice: Node.js with Express

- Reasoning: Excellent JSON processing performance, large ecosystem of libraries
- Benefits: Fast development, great for API services, strong community support
- Scalability: Handles concurrent connections efficiently through event-driven architecture

Database Selection

Primary Database: PostgreSQL

- Reasoning: Robust ACID compliance, excellent JSON support, proven scalability
- Offline Support: Works well with local SQLite instances on tablets
- Performance: Handles complex queries efficiently, supports partitioning for large datasets

Caching Layer: Redis

- Purpose: Fast access to frequently requested lists
- Benefits: In-memory performance, supports complex data structures

Frontend Employee Interface

Technology: React.js Progressive Web App

- Offline Capability: Service Workers cache application and data
- Performance: Virtual DOM provides smooth interactions on tablets
- Cross-Platform: Works on any tablet with a modern browser
- Local Storage: IndexedDB stores shopping lists.

Task 5: Interface Design

API endpoints:

- Get next available list: GET /api/lists/next
- Mark item collected/unavailable: POST /api/lists/{listId}/items/{itemId}
- Complete list collection: POST /api/lists/{listId}/complete
- Export payment data: GET /api/list/listId/payment-export
- Sync offline data: POST /api/sync/upload

Task 6: Data Volume and Performance

- Daily: $10,000 \times 500 \text{ KB} \approx 5 \text{ GB/day}$.
- Yearly: $5 \text{ GB} \times 365 \approx 1.8 \text{ TB/year}$ (rounded to $\sim 2 \text{ TB}$).

Scaling approach:

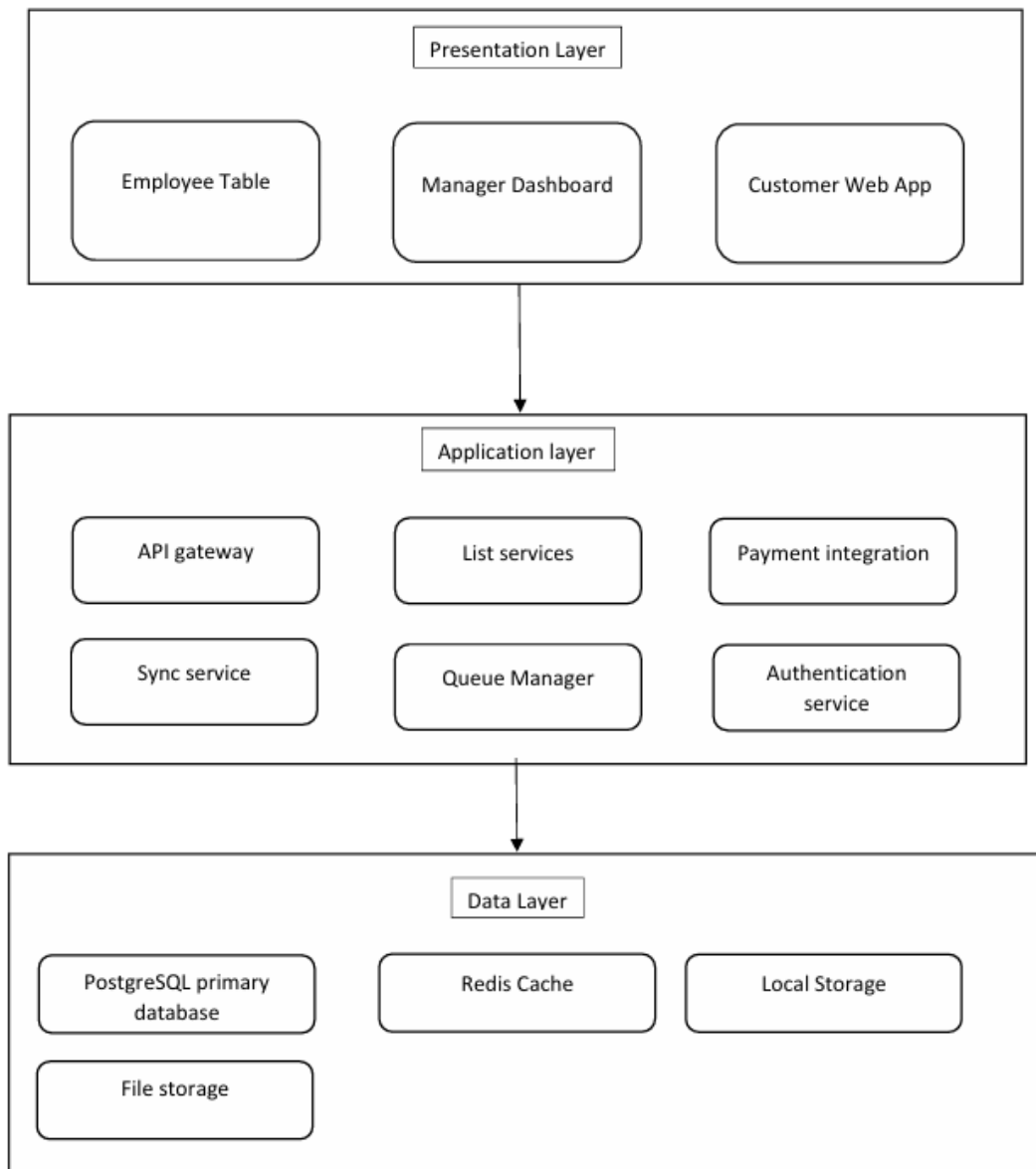
- Partition lists by date.
- Archive completed lists after 90 days to cloud storage.
- Use Redis for hot data.
- Horizontal scaling for APIs.
- Optimistic concurrency + versioning for sync conflicts.

Offline strategy:

- Tablets store operations locally with unique IDs.
- On reconnect, changes sync to server.
- Conflicts resolved via last-write-wins or supervisor intervention.

Task 7: Logical and Physical Diagrams

Logical Diagram



Physical Deployment Diagram

